

### **TCP server code:**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
int main() {
```

```
    int server_fd, client_fd;
```

```
    struct sockaddr_in server_addr, client_addr;
```

```
    socklen_t addr_size;
```

```
    char buffer[1024];
```

```
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (server_fd < 0) {
```

```
        perror("Socket creation failed");
```

```
        exit(1);
```

```
    }
```

```
    server_addr.sin_family = AF_INET;
```

```
    server_addr.sin_addr.s_addr = INADDR_ANY;
```

```
    server_addr.sin_port = htons(8080);
```

```
    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
```

```
        perror("Bind failed");
```

```
        exit(1);
```

```
}
```

```
listen(server_fd, 5);
```

```
printf("Server waiting for connection...\n");
```

```
addr_size = sizeof(client_addr);
```

```
client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);
```

```
recv(client_fd, buffer, sizeof(buffer), 0);
```

```
printf("Message from client: %s\n", buffer);
```

```
strcpy(buffer, "Hello from server");
```

```
send(client_fd, buffer, strlen(buffer) + 1, 0);
```

```
close(client_fd);
```

```
close(server_fd);
```

```
return 0;
```

```
}
```

# EXPLANATION OF SERVER CODE

## 1. Header Inclusions

- **<stdio.h> & <stdlib.h>**: Provide functions for standard input/output (printf, perror) and process control (exit).
- **<string.h>**: Provides string operations such as strcpy and strlen.
- **<unistd.h>**: Supplies POSIX operating system API functions, specifically close() for closing file descriptors.
- **<arpa/inet.h>**: Defines structures and functions for internet operations, including socket definitions and network byte ordering (htons).

## 2. State & Variable Initialization

Code:

```
int server_fd, client_fd;
```

```
struct sockaddr_in server_addr, client_addr;
```

```
socklen_t addr_size;
```

```
char buffer[1024];
```

- **server\_fd**: File descriptor referencing the primary socket set up to listen for incoming connection requests.
- **client\_fd**: File descriptor assigned to an active, individual client connection once accepted.
- **server\_addr & client\_addr**: IPv4-specific socket structures containing IP address family, port numbers, and interface bindings.
- **addr\_size**: Stores the byte size of the socket address structure.
- **buffer**: A 1024-byte character array used as a temporary data buffer for incoming and outgoing messages.

## 3. Socket Creation

Code:

```
server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
if (server_fd < 0) {
```

```
    perror("Socket creation failed");
```

```
    exit(1);
```

```
}
```

- **socket()**: Requests a new socket descriptor from the kernel.
  - **AF\_INET**: Designates IPv4 internet protocol family.
  - **SOCK\_STREAM**: Configures the socket for **TCP** (a connection-oriented, reliable, stream-based protocol).
  - **0**: Selects the default protocol for TCP (IPPROTO\_TCP).

- **Error Check:** If `socket()` returns a value less than 0, creation failed, an error message is printed, and execution halts.

#### 4. Address Setup & Binding

Code:

```
server_addr.sin_family = AF_INET;
```

```
server_addr.sin_addr.s_addr = INADDR_ANY;
```

```
server_addr.sin_port = htons(8080);
```

```
if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
```

```
    perror("Bind failed");
```

```
    exit(1);
```

```
}
```

- **INADDR\_ANY:** Binds the server to accept connections from any network interface on the host machine (e.g., localhost, local IP, or public IP).
- **htons(8080):** Converts the integer port 8080 from Host Byte Order to **Network Byte Order** (big-endian).
- **bind():** Registers the server socket with the assigned IP address and port number. Casting to `(struct sockaddr*)` matches the generic socket structure format required by POSIX system calls.

#### 5. Listening & Connection Acceptance

Code:

```
listen(server_fd, 5);
```

```
printf("Server waiting for connection...\n");
```

```
addr_size = sizeof(client_addr);
```

```
client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);
```

- **listen():** Switches the socket to passive mode to queue incoming connection requests. 5 defines the maximum pending connection backlog.
- **accept():** A **blocking system call** that suspends program execution until a client initiates a connection. Upon connection, it completes the TCP three-way handshake and returns a new socket handle (`client_fd`) dedicated exclusively to that client.

#### 6. Message Exchange

Code:

```
recv(client_fd, buffer, sizeof(buffer), 0);

printf("Message from client: %s\n", buffer);

strcpy(buffer, "Hello from server");

send(client_fd, buffer, strlen(buffer) + 1, 0);
```

- **recv()**: Reads the incoming payload sent by the client through client\_fd into buffer.
- **send()**: Writes the message "Hello from server" back to the connected client. Using strlen(buffer) + 1 ensures the null terminator (\0) is included so the string terminates properly on the client side.

## 7. Resource Cleanup

Code:

```
close(client_fd);

close(server_fd);

return 0;
```

- **close()**: Explicitly closes both file descriptors, closing the connection with the client and releasing port 8080 back to the operating system.

The screenshot shows a C code editor with a file named `main.c`. The code is a TCP server implementation. It starts by listening on a server file descriptor (`server_fd`) for 5 connections. When a connection is accepted, it prints a message, receives data from the client, prints the received message, and sends a response back to the client. Finally, it closes the client and server file descriptors and returns 0.

```
25     exit(1);
26 }
27
28 listen(server_fd, 5);
29 printf("Server waiting for connection...\n");
30
31 addr_size = sizeof(client_addr);
32 client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);
33
34 recv(client_fd, buffer, sizeof(buffer), 0);
35 printf("Message from client: %s\n", buffer);
36
37 strcpy(buffer, "Hello from server");
38 send(client_fd, buffer, strlen(buffer) + 1, 0);
39
40 close(client_fd);
41 close(server_fd);
42 return 0;
43 }
44
```

The output window shows the following messages:

```
Server waiting for connection...
Message from client: Hello from client

=== Code Execution Successful ===
```

Screenshot of tcp server code

### **TCP Client code:**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
int main() {
```

```
    int sockfd;
```

```
    struct sockaddr_in server_addr;
```

```
    char buffer[1024];
```

```
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (sockfd < 0) {
```

```
        perror("Socket creation failed");
```

```
        exit(1);
```

```
    }
```

```
    server_addr.sin_family = AF_INET;
```

```
    server_addr.sin_port = htons(8080);
```

```
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
```

```
    if (connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
```

```
        perror("Connection failed");
```

```
        exit(1);
```

```
    }
```

```
strcpy(buffer, "Hello from client");  
  
send(sockfd, buffer, strlen(buffer) + 1, 0);  
  
recv(sockfd, buffer, sizeof(buffer), 0);  
printf("Message from server: %s\n", buffer);  
  
close(sockfd);  
  
return 0;  
}
```



## EXPLANATION OF CLIENT CODE

### 1. Variable & Address Initialization

- The program allocates memory for two file descriptors: `server_fd` (for listening to overall traffic) and `client_fd` (for communicating with an active connection).
- `struct sockaddr_in` structures (`server_addr` and `client_addr`) hold the IP configuration parameters.
- A 1024-byte character array (`buffer`) acts as local memory to hold incoming and outgoing text.

### 2. Socket Creation (`socket`)

- `socket(AF_INET, SOCK_STREAM, 0)` requests an endpoint from the OS kernel.
- `AF_INET` sets IPv4 addressing, and `SOCK_STREAM` specifies **TCP** (a reliable, connection-oriented byte stream).

### 3. Address Binding (`bind`)

- The code configures the target address structure: `INADDR_ANY` tells the server to listen on all local network interfaces, and `htons(8080)` converts port 8080 to **Network Byte Order** (big-endian).
- `bind()` registers the socket to that IP/port combination. If another process is already using port 8080, it fails and exits.

### 4. Listening & Connection Handshake (`listen` & `accept`)

- `listen(server_fd, 5)` turns the socket into passive mode, allowing a queue of up to 5 pending connections.
- `accept()` is a **blocking call**. The server pauses here until a client connects. Once connected, `accept()` returns a brand-new socket file descriptor (`client_fd`) dedicated exclusively to communicating with that specific client.

### 5. Data Exchange (`recv` & `send`)

- `recv()` reads incoming raw bytes sent by the client into `buffer` and prints them.
- `send()` transmits the string "Hello from server" back across `client_fd`. `strlen(buffer) + 1` ensures the null terminator (`\0`) is sent so the client receives a clean string.

### 6. Cleanup (`close`)

- `close(client_fd)` closes the session with the connected client.
- `close(server_fd)` shuts down the server socket itself, freeing port 8080 back to the operating system.

main.c

Run ▶

```
17
18  server_addr.sin_family = AF_INET;
19  server_addr.sin_port = htons(8080);
20  server_addr.sin_addr.s_addr =
inet_addr("127.0.0.1");
21
22  if (connect(sockfd, (struct
sockaddr*)&server_addr, sizeof(server_addr))
< 0) {
23      perror("Connection failed");
24      exit(1);
25  }
26
27  strcpy(buffer, "Hello from client");
28  send(sockfd, buffer, strlen(buffer) +
1, 0);
29
30  recv(sockfd, buffer, sizeof(buffer), 0);
31  printf("Message from server: %s\n",
buffer);
```

Output

```
Message from server: Hello from server

=== Code Execution Successful ===
```

### UDP Server code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>


int main() {

    int sockfd;

    struct sockaddr_in server_addr, client_addr;

    socklen_t len;

    char buffer[1024];


    sockfd = socket(AF_INET, SOCK_DGRAM, 0);


    server_addr.sin_family = AF_INET;

    server_addr.sin_addr.s_addr = INADDR_ANY;

    server_addr.sin_port = htons(8080);


    bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));

    printf("UDP Server waiting...\n");


    len = sizeof(client_addr);

    recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr*)&client_addr, &len);

    printf("Client: %s\n", buffer);
```

```
strcpy(buffer, "Hello from UDP server");

sendto(sockfd, buffer, strlen(buffer) + 1, 0, (struct sockaddr*)&client_addr, len);


close(sockfd);

return 0;

}
```

## EXPLANATION OF SERVER CODE

### 1. Variable Setup

- `sockfd`: Holds a single file descriptor representing the UDP socket.
- `server_addr` & `client_addr`: IPv4 structures storing network metadata. `client_addr` acts as a container to automatically capture who sent incoming packets.
- `buffer`: A 1024-byte array for storing incoming and outgoing message payloads.

### 2. Datagram Socket Creation (`socket`)

Code:

```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

- **AF\_INET**: Specifies IPv4 protocol addressing.
- **SOCK\_DGRAM**: Configures the socket to use **UDP** (User Datagram Protocol) rather than TCP's stream mode.

### 3. Binding to IP & Port (`bind`)

Code:

```
server_addr.sin_family = AF_INET;
```

```
server_addr.sin_addr.s_addr = INADDR_ANY;
```

```
server_addr.sin_port = htons(8080);
```

```
bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
```

- `INADDR_ANY` configures the server to listen on all host network interfaces.
- `htons(8080)` converts port 8080 to Network Byte Order.
- `bind()` registers the UDP socket to port 8080 so the OS routes incoming datagrams directed at port 8080 to `sockfd`.

### 4. Receiving Datagrams (`recvfrom`)

Code:

```
len = sizeof(client_addr);
```

```
recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr*)&client_addr, &len);
```

- `recvfrom()` is a blocking call that pauses execution until a datagram arrives.
- **Crucial UDP Behavior**: Because there is no active connection, `recvfrom()` automatically populates `client_addr` with the sender's IP address and port number so the server knows where to send a reply.

### 5. Sending Replies (`sendto`)

Code:

```
strcpy(buffer, "Hello from UDP server");
```

```
sendto(sockfd, buffer, strlen(buffer) + 1, 0, (struct sockaddr*)&client_addr, len);
```

- `sendto()` dispatches the response datagram.
- It explicitly uses `client_addr` (captured by `recvfrom()`) to direct the packet back to the specific client that sent the message.

## **6. Cleanup (close)**

Code:

```
close(sockfd);
```

- Releases the socket descriptor and frees port 8080 back to the operating system.

main.c

Run ▶

14

15 server\_addr.sin\_family = AF\_INET;

16 server\_addr.sin\_addr.s\_addr = INADDR\_ANY;

17 server\_addr.sin\_port = htons(8080);

18

19 bind(sockfd, (struct sockaddr\*)&server\_addr,

sizeof(server\_addr));

20 printf("UDP Server waiting...\n");

21

22 len = sizeof(client\_addr);

23 recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct

sockaddr\*)&client\_addr, &len);

24 printf("Client: %s\n", buffer);

25

26 strcpy(buffer, "Hello from UDP server");

27 sendto(sockfd, buffer, strlen(buffer) + 1, 0, (struct

sockaddr\*)&client\_addr, len);

28

29 close(sockfd);

30 return 0;

Output

UDP Server waiting...  
Client: Hello from UDP client  
  
=== Code Execution Successful ===



### UDP Client code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>


int main() {

    int sockfd;

    struct sockaddr_in server_addr;

    socklen_t len;

    char buffer[1024];


    sockfd = socket(AF_INET, SOCK_DGRAM, 0);


    server_addr.sin_family = AF_INET;

    server_addr.sin_port = htons(8080);

    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");


    strcpy(buffer, "Hello from UDP client");

    len = sizeof(server_addr);


    sendto(sockfd, buffer, strlen(buffer) + 1, 0, (struct sockaddr*)&server_addr, len);

    recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr*)&server_addr, &len);

    printf("Server: %s\n", buffer);
```



```
close(sockfd);
```

```
return 0;
```

```
}
```

## EXPLANATION OF CLIENT CODE

### 1. Architectural Overview

This program implements a connectionless server using the User Datagram Protocol (UDP) in C. Unlike TCP, UDP does not require a connection setup or handshake (`listen()` or `accept()`); it simply binds to a port and processes isolated message packets (datagrams).

### 2. Functional Steps

- **Socket Allocation:** `socket(AF_INET, SOCK_DGRAM, 0)` creates a file descriptor (`sockfd`) configured for IPv4 datagram-based communications.
- **Address Binding:** Sets up `server_addr` to listen on all local network interfaces (`INADDR_ANY`) at port 8080 (converted to Network Byte Order via `htons()`). The `bind()` system call registers the socket with the operating system.
- **Receiving Datagrams:** `recvfrom()` blocks until an incoming packet arrives. It writes the message into buffer and automatically populates `client_addr` with the sender's IP address and port number.
- **Sending Response:** `strcpy()` copies "Hello from UDP server" into buffer, and `sendto()` transmits the packet back using the saved `client_addr` details.
- **Resource Cleanup:** `close(sockfd)` shuts down the socket descriptor and releases port 8080 back to the OS

```
main.c
Run ▶

12
13     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
14
15     server_addr.sin_family = AF_INET;
16     server_addr.sin_port = htons(8080);
17     server_addr.sin_addr.s_addr =
18         inet_addr("127.0.0.1");
19
20     strcpy(buffer, "Hello from UDP client");
21     len = sizeof(server_addr);
22
23     sendto(sockfd, buffer, strlen(buffer) + 1,
24             0, (struct sockaddr*)&server_addr, len);
25
26     recvfrom(sockfd, buffer, sizeof(buffer),
27              0, (struct sockaddr*)&server_addr, &len);
28     printf("Server: %s\n", buffer);
29
30     close(sockfd);
31     return 0;
32 }
```

```
Output
Server: Hello from UDP server

=== Code Execution Successful ===
```

=== Code Execution Successful ===